AI Gateway

Technology Stack Selection Document

Document Objective

This document defines the recommended technology stack for an AI Gateway system designed to maximize Large Language Model quality under cost and latency constraints.
The objective is to select appropriate programming languages, frameworks, infrastructure components, and operational tools while considering scalability, maintainability, performance, and AI ecosystem compatibility.

1 System Requirements

The AI Gateway should provide the following capabilities:

- Unified access to multiple Large Language Models.
- Intelligent model routing based on quality, cost, and latency.
- Support for optimization mechanisms such as summarization and verification.
- Request monitoring and cost tracking.
- Scalable deployment for enterprise workloads.
- Extensible architecture for future optimization algorithms.

2 Technology Stack Selection

2.1 Backend Programming Language

The backend language is responsible for implementing the gateway logic, routing policies, and communication with external AI services.

Technology	Advantages	Evaluation
Python	Extensive AI ecosystem, native support for ML libraries, rapid development	Recommended
Go	Excellent concurrency and low latency	Suitable for extremely high-scale systems
Java	Strong enterprise ecosystem and stability	Less common in AI-native applications

Selected Technology: Python
Reason:
Python provides the strongest integration with the AI ecosystem and allows rapid implementation of routing algorithms, optimization methods, and model orchestration components.

2.2 API Framework

Selected Technology: FastAPI
FastAPI is selected because AI Gateway workloads require asynchronous request handling, low latency communication, and easy integration with Python-based AI components.

Technology	Advantages	Evaluation
FastAPI	Asynchronous support, high performance, automatic API documentation	Recommended
Flask	Simple and mature framework	Limited scalability for asynchronous workloads
Go Fiber/Gin	Very high performance	Requires Go ecosystem

2.3 LLM Integration Layer

Technology	Advantages	Evaluation
LiteLLM	Unified interface for multiple LLM providers, OpenAI-compatible API	Recommended
LangChain	Useful for complex chains and agent workflows	Selective usage
Custom SDK Integration	Maximum flexibility	High maintenance cost

Selected Technology: LiteLLM

LiteLLM reduces integration complexity by providing a common interface for different LLM providers including commercial and self-hosted models.

2.4 Routing and Decision Engine

The routing engine is the core intelligence component of the gateway. Its objective is to select the optimal model according to:

- Expected response quality.
- Monetary cost.
- Latency constraints.
- Current system load.

Approach	Advantages	Evaluation
Custom Policy Engine	Allows implementation of project-specific optimization algorithms	Recommended
Rule-based Routing	Simple and interpretable	Baseline solution
ML-based Routing	Learns routing policies from historical data	Future extension

Selected Approach: Custom Routing Engine

Since the main contribution of the AI Gateway is optimizing quality-cost-latency trade-offs, the routing layer should remain fully customizable.

3 Infrastructure Components

The infrastructure layer provides the operational foundation required to deploy the AI Gateway in production environments. While the routing algorithm determines which model should process each request, the infrastructure components ensure that requests are processed reliably, efficiently, and at scale.

The selected technologies are evaluated based on scalability, availability, operational complexity, ecosystem maturity, and their compatibility with AI workloads. Since the gateway is expected to serve multiple users concurrently while interacting with several external LLM providers, infrastructure decisions have a direct impact on latency, throughput, and operational cost.

3.1 Caching Layer

Technology	Usage	Recommendation
Redis	Fast caching, semantic cache, rate limiting	Recommended
Memcached	Simple key-value caching	Limited usage

Selected Technology: Redis

Redis is selected because it provides significantly more functionality than a traditional key-value cache. Besides storing frequently requested responses, Redis can also maintain rate-limiting counters, session data, temporary routing decisions, and semantic cache indexes.

For an AI Gateway, semantic caching is particularly valuable because many user requests are semantically similar even when they are not identical. Instead of forwarding every request to an expensive LLM, the gateway can reuse previous responses when the semantic similarity exceeds a specified threshold. This reduces both inference latency and operational cost.

Redis also supports high-throughput concurrent access and integrates well with distributed deployments, making it suitable for horizontally scaled gateway instances.

3.2 Database

Technology	Usage	Recommendation
PostgreSQL	Logs, metadata, billing, analytics	Recommended
MongoDB	Flexible document storage	Alternative

Selected Technology: PostgreSQL

PostgreSQL is selected as the primary relational database due to its stability, mature ecosystem, and strong transactional guarantees. Although the gateway does not require complex relational queries for every request, it continuously generates valuable metadata such as routing decisions, user quotas, billing information, provider statistics, and historical performance metrics.

These data are essential for offline analysis, policy optimization, and future machine learning models that learn routing strategies from historical observations.

If large-scale analytical workloads become dominant, PostgreSQL can be extended with analytical databases such as ClickHouse without requiring major architectural changes.

3.3 Monitoring and Observability

Observability is one of the most important requirements for an AI Gateway because routing decisions must remain explainable and measurable. Every request should generate sufficient telemetry to understand why a particular model was selected and how that decision affected latency, quality, and operational cost.

A modern monitoring stack enables both real-time operational monitoring and long-term performance analysis.

Recommended stack:

- Prometheus for metric collection.
- Grafana for visualization.

- OpenSearch/ELK for centralized logging.

These components enable monitoring of:

- Request latency.
- Model utilization.
- Failure rates.
- Cost statistics.

Besides traditional infrastructure metrics, the gateway should monitor AI-specific indicators including token consumption, provider-specific latency distributions, routing frequencies, cache hit ratios, and model selection accuracy. These metrics provide valuable feedback for continuously improving routing policies.

3.4 Deployment Infrastructure

Technology	Advantages	Recommendation
Docker + Kubernetes	Scalable production deployment	Recommended
Docker Compose	Simple deployment	Small-scale systems only
Virtual Machines	Traditional deployment	Less flexible

Containerization provides portability and reproducibility across development, testing, and production environments. Docker simplifies the packaging of gateway services together with all required dependencies, while Kubernetes provides orchestration, automatic scaling, health checks, rolling updates, and failure recovery.

Since request volume in AI systems can fluctuate significantly, automatic horizontal scaling is particularly important. Kubernetes allows the gateway to dynamically allocate resources according to workload while maintaining high availability.

4 Final Recommended Technology Stack

Based on the previous evaluation, the following technology stack provides the best balance between implementation complexity, maintainability, scalability, and AI ecosystem compatibility. The selected components are widely adopted in modern production AI systems and provide sufficient flexibility for future extensions such as online learning, multi-model routing, and adaptive optimization.

Recommended Architecture

- Backend: Python + FastAPI
- LLM Abstraction: LiteLLM
- Routing Engine: Custom Quality-Cost-Latency Optimization Engine
- Cache: Redis
- Database: PostgreSQL
- Monitoring: Prometheus + Grafana
- Logging: OpenSearch / ELK
- Deployment: Docker + Kubernetes
- Local Model Serving: vLLM

5 Conclusion

The proposed technology stack establishes a solid foundation for developing an intelligent AI Gateway capable of optimizing response quality under explicit cost and latency constraints.

The selected technologies emphasize modularity, scalability, and ease of future extension. Python and FastAPI provide rapid development and seamless integration with modern AI frameworks, while infrastructure components such as Redis, PostgreSQL, Docker, and Kubernetes ensure that the system can be deployed reliably in production environments.

Furthermore, the modular architecture enables future research directions including adaptive routing policies, online learning, semantic caching, multi-model aggregation, uncertainty-aware verification, and dynamic budget optimization without requiring major architectural redesign.

Overall, this technology stack provides an effective balance between research flexibility and production readiness, making it suitable for both academic experimentation and enterprise-scale deployment.